

Looker Developer Guide

Power BI -> LookML build playbook for developers

Source PBIX: movie_rental_analysis.pbix

LookML model: movie_rental_analysis

Primary fact / explore: FactTable

Approach: deterministic Phase 2 (inventory-driven)

This PDF: full instructions, checks, suggestions, examples, and gaps

Inside this guide

- * Section 4: every PBI object type with step-by-step build instructions (one topic per page)
- * Section 5: concrete build plan for THIS PBIX (views, joins, measures, M)
- * Section 6: prioritized GAP list with actions
- * Sections 7-8: validation checklist and acceptance criteria

1. How to use this guide

- * Read Best practices first - mandatory.
- * Complete Prerequisites before editing LookML.
- * Read each Object equivalence page fully (Build + Check + Suggestions + Example).
- * Execute Build plan for this PBIX.
- * Close HIGH gaps (or get written sign-off).
- * Sign off using Validation and acceptance.

2. Best practices (mandatory)

Primary key on every view

Put primary_key: yes on the unique key. Required for symmetric aggregates.

Explicit join relationship

Always set relationship: many_to_one (or the correct cardinality). Never omit it.

Measures reference dimensions

Prefer sql: \${dimension} over raw \${TABLE}.col when the dimension exists.

Warehouse owns ETL

Power Query M is NOT LookML. Rebuild transforms in dbt/Dataform/SQL first.

Granular includes

Prefer include per view file (looker-skills modeling guidelines).

Explore descriptions

Every explore needs label + description.

No silent M:M / inactive joins

Review fan-out risk; use bridge tables or aliased joins intentionally.

No KPI claims without tests

Complex DAX TODOs are unfinished until side-by-side parity passes.

3. Prerequisites checklist

- * ☐ Looker project + Git branch ready
- * ☐ Admin connection created
- * ☐ Warehouse tables exist (or creation tickets filed)
- * ☐ Phase 1 inventory is for THIS PBIX
- * ☐ LOOKML_PROJECT.zip imported
- * ☐ SQL dialect known (BigQuery SAFE_DIVIDE vs Snowflake DIV0)

Do not start LookML polish until connection and warehouse tables are real. Placeholder
YOUR_LOOKER_CONNECTION / YOUR_PROJECT.YOUR_DATASET will fail validation.

4.1 Object equivalence

Table (business) -> view (.view.lkml)

What it means

Create views/<name>.view.lkml with ``view: <name> { sql_table_name: ... }``. Every view needs a `primary_key` dimension (Looker skills / symmetric aggregates).

How to build (follow these steps)

* 1. Create views/<name>.view.lkml with ``view: <name> { sql_table_name: ... }``. Every view needs a `primary_key` dimension (Looker skills / symmetric aggregates).

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/reference/param-view-view>

* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-view/SKILL.md>

4.2 Object equivalence

Column -> dimension (or dimension_group for dates)

What it means

Add ``dimension: field { type: ... sql: ${TABLE}.col ;; }``. Date/time columns -> ``dimension_group: ... { type: time timeframes: [raw, date, week, month, quarter, year] }``.

How to build (follow these steps)

* 1. Add ``dimension: field { type: ... sql: ${TABLE}.col ;; }``. Date/time columns -> ``dimension_group: ... { type: time timeframes: [raw, date, week, month, quarter, year] }``.

What to check

Suggestions / best practice

References

- * <https://cloud.google.com/looker/docs/reference/param-field-dimension>
- * <https://cloud.google.com/looker/docs/reference/param-field-dimension-group>

4.3 Object equivalence

Measure (DAX) -> measure

What it means

Map SUM/AVERAGE/COUNT/DISTINCTCOUNT to type: sum|average|count|count_distinct. Ratios -> type: number

+ SAFE_DIVIDE. CALCULATE/time-intel -> filters, period patterns, or warehouse + TODO.

How to build (follow these steps)

* 1. Map SUM/AVERAGE/COUNT/DISTINCTCOUNT to type: sum|average|count|count_distinct. Ratios -> type: number + SAFE_DIVIDE. CALCULATE/time-intel -> filters, period patterns, or warehouse + TODO.

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/reference/param-field-measure>

* <https://cloud.google.com/looker/docs/reference/field-reference>

4.4 Object equivalence

Relationship (M:1 From->To) -> explore join (relationship: many_to_one)

What it means

In the model file, `explore: fact { join: dim { type: left\_outer relationship: many\_to\_one sql\_on: \${fact.fk} = \${dim.pk} ;; } }`. Always set relationship explicitly (looker-skills).

How to build (follow these steps)

* 1. In the model file, `explore: fact { join: dim { type: left\_outer relationship: many\_to\_one sql\_on: \${fact.fk} = \${dim.pk} ;; } }`. Always set relationship explicitly (looker-skills).

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/reference/param-explore-join>

* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-explore/SKILL.md>

4.5 Object equivalence

Calculated column (DAX) -> dimension (prefer warehouse column)

What it means

Prefer materializing in warehouse SQL, then expose as dimension. Simple row expressions may use LookML sql;; complex DAX stays as migration TODO.

How to build (follow these steps)

* 1. Prefer materializing in warehouse SQL, then expose as dimension. Simple row expressions may use LookML sql;; complex DAX stays as migration TODO.

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>

4.6 Object equivalence

Calculated table -> view (sql_table_name or derived_table)

What it means

If the table is seeded/small, warehouse seed + standard view. Otherwise SQL derived table / NDT per lookml-view derived_table guidance.

How to build (follow these steps)

* 1. If the table is seeded/small, warehouse seed + standard view. Otherwise SQL derived table / NDT per lookml-view derived_table guidance.

What to check

Suggestions / best practice

References

* <https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-view/SKILL.md>

4.7 Object equivalence

Power Query M -> Warehouse / ETL (not LookML)

What it means

Rebuild M transforms in the warehouse (dbt/Dataform/SQL). LookML only points sql_table_name at the finished table.

How to build (follow these steps)

* 1. Rebuild M transforms in the warehouse (dbt/Dataform/SQL). LookML only points sql_table_name at the finished table.

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>

4.8 Object equivalence

Hierarchy -> drill_fields / sets / dimension_group timeframes

What it means

Date hierarchies -> dimension_group timeframes. Attribute hierarchies -> drill_fields: [year, quarter, month, day] or sets.

How to build (follow these steps)

* 1. Date hierarchies -> dimension_group timeframes. Attribute hierarchies -> drill_fields: [year, quarter, month, day] or sets.

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/reference/param-field-drill-fields>

4.9 Object equivalence

RLS / OLS -> access_grant / access_filter / required_access_grants

What it means

Map roles to Looker user attributes + access_filter on explores, or access_grant on fields.

How to build (follow these steps)

* 1. Map roles to Looker user attributes + access_filter on explores, or access_grant on fields.

What to check

Suggestions / best practice

References

*

<https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-access-grants/SKILL.md>

4.10 Object equivalence

LocalDateTable_* / DataTableTemplate_* -> Skip (use business date + dimension_group)

What it means

Do not migrate auto-date tables. Use the business date column with Looker timeframes.

How to build (follow these steps)

* 1. Do not migrate auto-date tables. Use the business date column with Looker timeframes.

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/reference/param-field-dimension-group>

4.11 Object equivalence

Model / Dataset -> model (.model.lkml) + connection

What it means

One model file: connection, includes, datagroup, explore(s). Prefer granular includes over wildcards (lookml-modeling-guidelines).

How to build (follow these steps)

* 1. One model file: connection, includes, datagroup, explore(s). Prefer granular includes over wildcards (lookml-modeling-guidelines).

What to check

Suggestions / best practice

References

* <https://cloud.google.com/looker/docs/lookml-terms-and-concepts>

<https://github.com/looker-open-source/looker-skills/blob/main/skills/lookml-modeling-guidelines/SK>

* ILL.md

5. Build plan for this PBIX

Step A - Import LookML ZIP

- * Unzip LOOKML_PROJECT.zip into the Looker project.
- * Open models/<model>.model.lkml and set connection.
- * Run LookML Validator (expect table errors until Step B).

Step B - Point each view at the warehouse

For each view: set sql_table_name, confirm PK and columns exist.

View `actor` (Power BI: actor)

File: views/actor.view.lkml | columns: 4 | PK guess: actor_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `address` (Power BI: address)

File: views/address.view.lkml | columns: 9 | PK guess: address_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `category` (Power BI: category)

File: views/category.view.lkml | columns: 3 | PK guess: category_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `city` (Power BI: city)

File: views/city.view.lkml | columns: 4 | PK guess: city_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `country` (Power BI: country)

File: views/country.view.lkml | columns: 3 | PK guess: country_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `customer` (Power BI: customer)

File: views/customer.view.lkml | columns: 10 | PK guess: customer_id | date cols: create_date, last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `film` (Power BI: film)

File: views/film.view.lkml | columns: 13 | PK guess: film_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `film\_actor` (Power BI: film_actor)

File: views/film_actor.view.lkml | columns: 3 | PK guess: actor_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `film\_category` (Power BI: film_category)

File: views/film_category.view.lkml | columns: 3 | PK guess: film_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `film\_text` (Power BI: film_text)

File: views/film_text.view.lkml | columns: 3 | PK guess: film_id | date cols: none

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `inventory` (Power BI: inventory)

File: views/inventory.view.lkml | columns: 4 | PK guess: inventory_id | date cols: last_update

- * Confirm primary_key matches warehouse uniqueness.
- * Map dates to dimension_group timeframes.

View `language` (Power BI: language)

File: views/language.view.lkml | columns: 3 | PK guess: language_id | date cols: last_update

* Confirm primary_key matches warehouse uniqueness.

* Map dates to dimension_group timeframes.

View `payment` (Power BI: payment)

File: views/payment.view.lkml | columns: 7 | PK guess: payment_id | date cols: payment_date, last_update

* Confirm primary_key matches warehouse uniqueness.

* Map dates to dimension_group timeframes.

View `rentat` (Power BI: rentat)

File: views/rentat.view.lkml | columns: 7 | PK guess: rental_id | date cols: rental_date, return_date, last_update

* Confirm primary_key matches warehouse uniqueness.

* Map dates to dimension_group timeframes.

View `staff` (Power BI: staff)

File: views/staff.view.lkml | columns: 11 | PK guess: staff_id | date cols: last_update

* Confirm primary_key matches warehouse uniqueness.

* Map dates to dimension_group timeframes.

View `store` (Power BI: store)

File: views/store.view.lkml | columns: 4 | PK guess: store_id | date cols: last_update

* Confirm primary_key matches warehouse uniqueness.

* Map dates to dimension_group timeframes.

Step C - Explores and joins

Primary explore base: FactTable. Set type + relationship + sql_on on every join.

film_actor[actor_id] -> actor[actor_id] [M:1, ACTIVE]

Looker action: join actor relationship from PBI M:1

address[city_id] -> city[city_id] [M:1, ACTIVE]

Looker action: join city relationship from PBI M:1

city[country_id] -> country[country_id] [M:1, ACTIVE]

Looker action: join country relationship from PBI M:1

film[language_id] -> language[language_id] [M:1, ACTIVE]

Looker action: join language relationship from PBI M:1

film_category[category_id] -> category[category_id] [M:1, ACTIVE]

Looker action: join category relationship from PBI M:1

film[film_id] -> film_category[film_id] [M:1, ACTIVE]

Looker action: join film_category relationship from PBI M:1

film[film_id] -> film_text[film_id] [M:1, ACTIVE]

Looker action: join film_text relationship from PBI M:1

store[store_id] -> staff[store_id] [M:1, ACTIVE]

Looker action: join staff relationship from PBI M:1

film_actor[film_id] -> film[film_id] [M:1, ACTIVE]

Looker action: join film relationship from PBI M:1

customer[address_id] -> address[address_id] [M:1, ACTIVE]

Looker action: join address relationship from PBI M:1

payment[customer_id] -> customer[customer_id] [M:1, ACTIVE]

Looker action: join customer relationship from PBI M:1

customer[store_id] -> store[store_id] [M:1, ACTIVE]

Looker action: join store relationship from PBI M:1

rentat[inventory_id] -> inventory[inventory_id] [M:1, ACTIVE]

Looker action: join inventory relationship from PBI M:1

rentat[rental_id] -> payment[rental_id] [M:1, ACTIVE]

Looker action: join payment relationship from PBI M:1

rentat[staff_id] -> staff[staff_id] [M:1, INACTIVE]

Looker action: join staff relationship from PBI M:1

* Keep aliased; do not use as default path until a measure needs it.

inventory[film_id] -> film[film_id] [M:1, ACTIVE]

Looker action: join film relationship from PBI M:1

Step D - Measures

Mapped measures: 5. TODOs: 2.

- * [OK] payment.Revenue - strategy=direct_sum
- * [TODO] inventory.Total Films Rented - strategy=complex_todo
- * [OK] inventory.Average Inventory Value - strategy=direct_average
- * [OK] inventory.Inventory Turnover Rate - strategy=ratio
- * [TODO] actor.FilmPopularity - strategy=complex_todo

Step E - Calculated columns (build in warehouse)

1 calculated columns - materialize then expose as dimensions.

- * customer.Employment Duration: DATEDIFF('customer'[create_date], TODAY(), YEAR)

Step F - Power Query (warehouse, not LookML)

- * Rebuild query `actor` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `address` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `city` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `country` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `category` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `film` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `film\_actor` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `film\_category` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `film\_text` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `inventory` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `language` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `payment` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `rentat` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `staff` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `store` (source_type=file) in ETL; then point sql_table_name.
- * Rebuild query `customer` (source_type=file) in ETL; then point sql_table_name.

6. Gaps - prioritize and close

Each gap blocks production readiness until resolved or formally accepted.

Severity HIGH (4)

GAP [HIGH] - Connection & warehouse

What is missing: LookML still uses placeholders YOUR_LOOKER_CONNECTION and YOUR_PROJECT.YOUR_DATASET.

What to do:

Set the real Looker connection name and point every sql_table_name at existing warehouse tables before validating.

GAP [HIGH] - Power Query / ETL

What is missing: 16 Power Query queries exist in PBIX. LookML does not recreate M.

What to do:

Rebuild each query's grain and transforms in the warehouse (dbt/Dataform/SQL). Confirm row counts and keys match Power BI before Looker go-live. Queries: actor, address, city, country, category, film, film_actor, film_category, film_text, inventory, language, payment, rental, staff, store, customer

GAP [HIGH] - Complex DAX

What is missing: 2 of 5 measures are TODO stubs (CALCULATE/time-intel/iterators).

What to do:

Implement each TODO using LookML filters, period-over-period patterns, or warehouse logic. Keep original DAX in the field description until KPI parity passes.

GAP [HIGH] - Calculated columns

What is missing: 1 business calculated columns must be materialized (prefer warehouse), not left as DAX.

What to do:

Create warehouse columns for: customer.Employment Duration

Severity MEDIUM (1)

GAP [MEDIUM] - Inactive relationship

What is missing: Inactive in Power BI: rental[staff_id] -> staff[staff_id] (M:1).

What to do:

Looker has no inactive join. Keep as a separate aliased join (from:.) and only expose when a measure needs USERRELATIONSHIP-style behavior. Hide fields until needed.

Severity LOW (2)

GAP [LOW] - Row-level security

What is missing: No RLS roles in this PBIX inventory.

What to do:

Still confirm with security owners whether Looker needs access_filter by region/org.

GAP [LOW] - Auto date tables

What is missing: 20 LocalDateTable_/DateTableTemplate_ tables skipped (correct).

What to do:

Use business date columns with dimension_group timeframes. Do not migrate auto-date tables.

7. What to check (validation)

LookML compile

- * ☐ Validator: 0 errors
- * ☐ One primary_key per view
- * ☐ Every join has relationship + sql_on
- * ☐ Dialect-correct SQL

Grain and joins

- * ☐ Fact row count matches Power BI (same filters)
- * ☐ count vs count_distinct(PK) shows no unexpected fan-out
- * ☐ Orphan FK rate understood for left_outer joins

Business logic

- * ☐ All HIGH gaps closed or signed off
- * ☐ Calculated columns available
- * ☐ Required measures implemented and spot-checked vs Power BI
- * ☐ Inactive / M:M joins reviewed

Security and ops

- * ☐ RLS / access_filter decision recorded
- * ☐ Caching / datagroup set
- * ☐ QA explores hidden from end users

8. Acceptance criteria

- * Validator clean on target connection
- * HIGH gaps closed or formally accepted
- * At least 5 Power BI business questions reproduce in Looker within agreed tolerance
- * Remaining MEDIUM/LOW gaps have owners and dates
- * Do NOT publish claiming full KPI parity while DAX TODOs remain

9. Inventory snapshot

- * tables: 36
- * business_tables: 16
- * internal_tables: 20
- * columns: 231
- * measures: 5
- * calculated_columns: 121
- * calculated_tables: 20
- * relationships: 16
- * power_query: 16
- * m_files: 16
- * rls_roles: 0
- * hierarchies: 20
- * partitions: 322
- * auto_date_tables: 20
- * annotations: 433
- * sort_by_columns: 0
- * format_strings: 63
- * perspectives: 0
- * display_folders: 0

10. Out of scope

- * Power BI report pages, visuals, bookmarks, themes
- * Automatic warehouse DDL generation
- * Automatic KPI certification

Generated by phase2/generate_developer_guide.py for movie_rental_analysis.pbix.